

Databricks Administration — Hands-On Lab

Four practical admin tasks with UI + CLI/JSON/SQL

cluster policy · secret scope · Unity Catalog grants · scheduled job

Four practical admin tasks you can actually do, each with **UI steps** and the **CLI/JSON/SQL** equivalent (which is how real admins automate). Do them in order.

Prereqs

- Admin (or sufficient) access to a Databricks workspace. (Unity Catalog grants need a UC-enabled workspace; the rest works on any workspace, incl. much of Community Edition via UI.)
- **Databricks CLI** for the code path:

```
pip install databricks-cli          # or the newer 'databricks' CLI
databricks configure --token         # enter workspace URL + a personal access token
# token: User Settings → Developer → Access tokens → Generate
```

LAB 1 — Create a Cluster Policy (control cost & standardize clusters)

A **cluster policy** limits what clusters users can create (size, auto-termination, node types) — the #1 cost-control lever.

UI: Compute → **Policies** → **Create policy** → paste the JSON → Create → then let users create clusters "From policy."

Policy JSON (`policy.json`) — forces auto-termination, caps workers, restricts node type:

```
{
  "spark_version":      { "type": "fixed", "value": "14.3.x-scala2.12" },
  "autotermination_minutes": { "type": "fixed", "value": 30 },
  "node_type_id":       { "type": "allowlist", "values": ["m5d.large", "m5d.xlarge"] },
  "num_workers":        { "type": "range", "maxValue": 8 },
  "custom_tags.team":    { "type": "fixed", "value": "data-eng" },
  "spark_conf.spark.databricks.cluster.profile": { "type": "forbidden" }
}
```

CLI:

```
databricks cluster-policies create --json @policy.json
databricks cluster-policies list
```

Why it matters: auto-termination stops idle clusters (huge savings), the worker cap prevents runaway cost, allowlisted node types standardize config, and the `team` tag enables cost attribution.

LAB 2 — Create a Secret Scope and store a credential

Never hard-code passwords/keys. Store them in a **secret scope** and read at runtime.

CLI:

```
# 1) create a scope
databricks secrets create-scope --scope data-eng

# 2) put a secret (opens an editor to paste the value, or use --string-value)
databricks secrets put-secret --scope data-eng --key db_password --string-value 'S3cr3t!'

# 3) verify
databricks secrets list-scopes
databricks secrets list-secrets --scope data-eng
```

Use it in a notebook (value is redacted in output):

```
pwd = dbutils.secrets.get(scope="data-eng", key="db_password")
# e.g. jdbc connect:
df = (spark.read.format("jdbc")
      .option("url","jdbc:postgresql://host:5432/db")
      .option("user","app").option("password", pwd)
      .option("dbtable","public.orders").load())
```

Grant access to a group (ACL):

```
databricks secrets put-acl --scope data-eng --principal group_data_eng --permission READ
```

Why: secrets stay out of code/Git; access is controlled per group; values are masked in logs.

LAB 3 — Unity Catalog: create objects and GRANT access

Set up governed data and grant least-privilege access. Run the **SQL** in a SQL editor or a notebook cell (`%sql`).

```

-- 1) hierarchy: catalog -> schema -> table
CREATE CATALOG IF NOT EXISTS sales;
CREATE SCHEMA IF NOT EXISTS sales.core;

-- 2) a managed table (or point at your Parquet/Delta)
CREATE TABLE IF NOT EXISTS sales.core.orders (
  order_id BIGINT, customer_id BIGINT, amount DOUBLE, order_date DATE
) USING DELTA;

-- 3) least-privilege grants to GROUPS (not individuals)
GRANT USE CATALOG ON CATALOG sales TO `analysts`;
GRANT USE SCHEMA ON SCHEMA sales.core TO `analysts`;
GRANT SELECT ON TABLE sales.core.orders TO `analysts`;

-- engineers can write
GRANT MODIFY, SELECT ON TABLE sales.core.orders TO `data_engineers`;

-- 4) check + revoke
SHOW GRANTS ON TABLE sales.core.orders;
-- REVOKE SELECT ON TABLE sales.core.orders FROM `analysts`;

```

Column/row security (advanced):

```

-- mask a sensitive column for non-privileged users via a dynamic view
CREATE VIEW sales.core.orders_secure AS
SELECT order_id, customer_id,
  CASE WHEN is_account_group_member('finance') THEN amount ELSE NULL END AS amount,
  order_date
FROM sales.core.orders;
GRANT SELECT ON VIEW sales.core.orders_secure TO `analysts`;

```

Why: central governance, table/column/row control, automatic lineage + audit; users query by name (`catalog.schema.table`), never by path.

LAB 4 — Create a Scheduled Job (with a job cluster)

Automate a notebook to run on a schedule using a **job cluster** (spins up per run — cheaper than an always-on cluster) and retries.

UI: Workflows → **Create Job** → Task: notebook path → **New job cluster** → set **Schedule** (cron) → add **retries** and **email alerts** → Create.

CLI/JSON (`job.json`):

```
{
  "name": "daily-orders-etl",
  "schedule": { "quartz_cron_expression": "0 0 6 * * ?", "timezone_id": "UTC" },
  "email_notifications": { "on_failure": ["you@example.com"] },
  "max_concurrent_runs": 1,
  "tasks": [
    {
      "task_key": "run_etl",
      "notebook_task": { "notebook_path": "/Repos/you/de/databricks_de_practice" },
      "new_cluster": {
        "spark_version": "14.3.x-scala2.12",
        "node_type_id": "m5d.large",
        "num_workers": 2,
        "autotermination_minutes": 30,
        "custom_tags": { "team": "data-eng" }
      },
      "max_retries": 2,
      "min_retry_interval_millis": 60000
    }
  ]
}
```

```
databricks jobs create --json @job.json
databricks jobs list
databricks jobs run-now --job-id <ID>          # trigger a manual run
```

Why: job clusters + schedule + retries + alerts = reliable, cheap automation. Chain multiple tasks (ingest → transform → quality) into a DAG with dependencies.

Bonus admin quick-wins

- **SQL Warehouse for BI:** SQL → Warehouses → Create → **Serverless**, set **auto-stop** (e.g., 10 min). Analysts run dashboards without touching clusters.
- **Cluster pool:** Compute → Pools → create a pool of pre-warmed nodes → point job clusters at it to cut startup time.
- **Cost tags:** enforce `custom_tags` via the cluster policy (Lab 1) so all spend is attributable.
- **Auto-termination everywhere:** the single biggest cost saver — enforce it in the policy.

- **Monitoring:** enable **job failure alerts**; review **Spark UI** for slow stages; use system tables / usage dashboards for DBU spend.
-

What you practiced (admin talking points)

- **Cluster policies** to standardize + cap cost (auto-termination, worker limits, node allowlist, tags).
- **Secret scopes** to keep credentials out of code, with group ACLs.
- **Unity Catalog** governance — catalog/schema/table, least-privilege **GRANTs**, column/row security, lineage/audit.
- **Scheduled Jobs** with **job clusters**, retries, and alerts — reliable, cost-efficient automation.

(These four are exactly what a Databricks admin/platform interview asks you to have done.)